



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 2

на тему:

«Решение непрерывной выпукло-вогнутой антагонистической
игры аналитическим и численным методами»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н.С. Коннова
(И.О. Фамилия)

Москва, 2026 г.

Цель работы:

Найти оптимальные стратегии непрерывной выпукло-вогнутой антагонистической игры аналитическим и численным методами.

Постановка задачи:

Рассматривается непрерывная антагонистическая игра на единичном квадрате

$$0 \leq x \leq 1, \quad 0 \leq y \leq 1$$

с функцией выигрыша

$$H(x, y) = ax^2 + by^2 + cxy + dx + ey.$$

Требуется:

1. Найти аналитическое решение игры;
2. Получить численное решение методом аппроксимации на сетке;
3. На каждом шаге строить матрицу $H^{(N)}$, искать седловую точку;
4. Если седловой точки нет, применять метод Брауна-Робинсон;
5. Увеличивать N до выполнения критерия останова.

Исходные данные:

Для варианта 13:

$$a = -4, b = 2, c = 8, d = -\frac{4}{5}, e = -\frac{32}{5}$$

Следовательно,

$$H(x, y) = -4x^2 + 2y^2 + 8xy - \frac{4}{5}x - \frac{32}{5}y$$

Ход работы:Аналитическое решение

Проверим условия выпукло-вогнутости:

$$H_{xx} = 2a = -8 < 0, \quad H_{yy} = 2b = 4 > 0$$

Следовательно, игра является выпукло-вогнутой и разрешима в чистых стратегиях.

Находим частные производные и приравняем их к нулю

$$H_x = -8x + 8y - \frac{4}{5} = 0, \quad H_y = 4y + 8x - \frac{32}{5} = 0$$

Решая систему, получаем:

$$x^* = \frac{1}{2} = 0.5, \quad y^* = \frac{3}{5} = 0.6$$

Найдём цену игры:

$$H\left(\frac{1}{2}, \frac{3}{5}\right) = -4\left(\frac{1}{2}\right)^2 + 2\left(\frac{3}{5}\right)^2 + 8 \cdot \frac{1}{2} \cdot \frac{3}{5} - \frac{4}{5} \cdot \frac{1}{2} - \frac{32}{5} \cdot \frac{3}{5} = -\frac{53}{25} = -2.12$$

Итоговое аналитическое решение:

$$x^* = 0.5, \quad y^* = 0.6, \quad h = -2.12$$

Численное решение

Для численного решения использовалась аппроксимация функции выигрыша на сетке:

$$H^{(N)} = \left(H_{ij}^{(N)}\right), \quad H_{ij}^{(N)} = H\left(\frac{i}{N}, \frac{j}{N}\right), \quad i, j = 0, \dots, N.$$

На каждом шаге:

- строилась матрица размера $(N + 1) \times (N + 1)$;
- проверялось наличие седловой точки;
- при её отсутствии применялся метод Брауна-Робинсон;
- сравнивались значения H_k и H_{k-1}

Критерий останова:

$$|H_k - H_{k-1}| \leq 10^{-5}$$

Окончательный результат получен на 364-й итерации решения, что соответствует матрице размера 366×366 .

По этому для отчёта целесообразно привести только первые 10 итераций численного метода.

Сводная таблица за первые 10 итераций численного метода:

k	(N)	Размер матрицы	Метод	(x)	(y)	(H)	Δ_{prev}
1	2	3x3	седло	0.500000	0.500000	-2.100000	—
2	3	4x4	Браун-Робинсон	0.511737	0.619718	-2.117922	0.017922
3	4	5x5	седло	0.500000	0.500000	-2.100000	0.017922
4	5	6x6	Браун-Робинсон	0.459155	0.588732	-2.122738	0.022738
5	6	7x7	седло	0.500000	0.666667	-2.111111	0.011626
6	7	8x8	Браун-Робинсон	0.474026	0.586580	-2.119550	0.008439
7	8	9x9	седло	0.500000	0.625000	-2.118750	0.000800
8	9	10x10	Браун-Робинсон	0.487179	0.584615	-2.118606	0.000144
9	10	11x11	седло	0.500000	0.600000	-2.120000	0.001394
10	11	12x12	Браун-Робинсон	0.490397	0.587708	-2.119122	0.000878

Подробный результат первых 10 итераций численного решения представлен ниже.

```

-----
Итерация 1
Разбиение: N = 2, матрица 3x3
Матрица игры:
    0.000000    -2.700000    -4.400000
   -1.400000    -2.100000    -1.800000
   -4.800000    -3.500000    -1.200000
Метод решения: седловая точка
x = 0.500000 = 1/2
y = 0.500000 = 1/2
H = -2.100000 = -21/10
Нижняя цена игры max(min) = -2.100000
Верхняя цена игры min(max) = -2.100000
Δ с предыдущим приближением: ---
Седловые точки: (2, 2)
-----

```

Рисунок 1 – итерация численного решения №1

Итерация 2
Разбиение: $N = 3$, матрица 4×4
Матрица игры:

0.000000	-1.911111	-3.377778	-4.400000
-0.711111	-1.733333	-2.311111	-2.444444
-2.311111	-2.444444	-2.133333	-1.377778
-4.800000	-4.044444	-2.844444	-1.200000

Метод решения: Браун-Робинсон
 $x = 0.511737 = 109/213$
 $y = 0.619718 = 44/71$
 $N = -2.117922 = -7705/3638$
Нижняя цена игры $\max(\min) = -2.444444$
Верхняя цена игры $\min(\max) = -2.133333$
 Δ с предыдущим приближением: 0.017922
Итераций Брауна-Робинсон: 71
Верхняя оценка цены игры: -2.142222
Нижняя оценка цены игры: -2.241002
Погрешность Брауна-Робинсон: 0.098779

Рисунок 2 – итерация численного решения №2

Итерация 3
Разбиение: $N = 4$, матрица 5×5
Матрица игры:

0.000000	-1.475000	-2.700000	-3.675000	-4.400000
-0.450000	-1.425000	-2.150000	-2.625000	-2.850000
-1.400000	-1.875000	-2.100000	-2.075000	-1.800000
-2.850000	-2.825000	-2.550000	-2.025000	-1.250000
-4.800000	-4.275000	-3.500000	-2.475000	-1.200000

Метод решения: седловая точка
 $x = 0.500000 = 1/2$
 $y = 0.500000 = 1/2$
 $N = -2.100000 = -21/10$
Нижняя цена игры $\max(\min) = -2.100000$
Верхняя цена игры $\min(\max) = -2.100000$
 Δ с предыдущим приближением: 0.017922
Седловые точки: (3, 3)

Рисунок 3 – итерация численного решения №3

Итерация 4
Разбиение: $N = 5$, матрица 6×6
Матрица игры:

0.000000	-1.200000	-2.240000	-3.120000	-3.840000	-4.400000
-0.320000	-1.200000	-1.920000	-2.480000	-2.880000	-3.120000
-0.960000	-1.520000	-1.920000	-2.160000	-2.240000	-2.160000
-1.920000	-2.160000	-2.240000	-2.160000	-1.920000	-1.520000
-3.200000	-3.120000	-2.880000	-2.480000	-1.920000	-1.200000
-4.800000	-4.400000	-3.840000	-3.120000	-2.240000	-1.200000

Метод решения: Браун-Робинсон
 $x = 0.459155 = 163/355$
 $y = 0.588732 = 209/355$
 $N = -2.122738 = -3753/1768$
Нижняя цена игры $\max(\min) = -2.240000$
Верхняя цена игры $\min(\max) = -2.160000$
 Δ с предыдущим приближением: 0.022738
Итераций Брауна-Робинсон: 71
Верхняя оценка цены игры: -2.101408
Нижняя оценка цены игры: -2.200563
Погрешность Брауна-Робинсон: 0.099155

Рисунок 4 – итерация численного решения №4

```

-----
Итерация 5
Разбиение: N = 6, матрица 7x7
Матрица игры:
  0.000000  -1.011111  -1.911111  -2.700000  -3.377778  -3.944444  -4.400000
 -0.244444  -1.033333  -1.711111  -2.277778  -2.733333  -3.077778  -3.311111
 -0.711111  -1.277778  -1.733333  -2.077778  -2.311111  -2.433333  -2.444444
 -1.400000  -1.744444  -1.977778  -2.100000  -2.111111  -2.011111  -1.800000
 -2.311111  -2.433333  -2.444444  -2.344444  -2.133333  -1.811111  -1.377778
 -3.444444  -3.344444  -3.133333  -2.811111  -2.377778  -1.833333  -1.177778
 -4.800000  -4.477778  -4.044444  -3.500000  -2.844444  -2.077778  -1.200000

Метод решения: седловая точка
x = 0.500000 = 1/2
y = 0.666667 = 2/3
N = -2.111111 = -19/9
Нижняя цена игры max(min) = -2.111111
Верхняя цена игры min(max) = -2.111111
Δ с предыдущим приближением: 0.011626
Седловые точки: (4, 5)
-----

```

Рисунок 5 – итерация численного решения №5

```

-----
Итерация 6
Разбиение: N = 7, матрица 8x8
Матрица игры:
  0.000000  -0.873469  -1.665306  -2.375510  -3.004082  -3.551020  -4.016327  -4.400000
 -0.195918  -0.906122  -1.534694  -2.081633  -2.546939  -2.930612  -3.232653  -3.453061
 -0.555102  -1.102041  -1.567347  -1.951020  -2.253061  -2.473469  -2.612245  -2.669388
 -1.077551  -1.461224  -1.763265  -1.983673  -2.122449  -2.179592  -2.155102  -2.048980
 -1.763265  -1.983673  -2.122449  -2.179592  -2.155102  -2.048980  -1.861224  -1.591837
 -2.612245  -2.669388  -2.644898  -2.538776  -2.351020  -2.081633  -1.730612  -1.297959
 -3.624490  -3.518367  -3.330612  -3.061224  -2.710204  -2.277551  -1.763265  -1.167347
 -4.800000  -4.530612  -4.179592  -3.746939  -3.232653  -2.636735  -1.959184  -1.200000

Метод решения: Браун-Робинсон
x = 0.474026 = 73/154
y = 0.586580 = 271/462
N = -2.119550 = -1507/711
Нижняя цена игры max(min) = -2.179592
Верхняя цена игры min(max) = -2.122449
Δ с предыдущим приближением: 0.008439
Итераций Брауна-Робинсона: 66
Верхняя оценка цены игры: -2.085220
Нижняя оценка цены игры: -2.185034
Погрешность Брауна-Робинсона: 0.099814
-----

```

Рисунок 6 – итерация численного решения №6

```

-----
Итерация 7
Разбиение: N = 8, матрица 9x9
Матрица игры:
  0.000000  -0.768750  -1.475000  -2.118750  -2.700000  -3.218750  -3.675000  -4.068750  -4.400000
 -0.162500  -0.806250  -1.387500  -1.906250  -2.362500  -2.756250  -3.087500  -3.356250  -3.562500
 -0.450000  -0.968750  -1.425000  -1.818750  -2.150000  -2.418750  -2.625000  -2.768750  -2.850000
 -0.862500  -1.256250  -1.587500  -1.856250  -2.062500  -2.206250  -2.287500  -2.306250  -2.262500
 -1.400000  -1.668750  -1.875000  -2.018750  -2.100000  -2.118750  -2.075000  -1.968750  -1.800000
 -2.062500  -2.206250  -2.287500  -2.306250  -2.262500  -2.156250  -1.987500  -1.756250  -1.462500
 -2.850000  -2.868750  -2.825000  -2.718750  -2.550000  -2.318750  -2.025000  -1.668750  -1.250000
 -3.762500  -3.656250  -3.487500  -3.256250  -2.962500  -2.606250  -2.187500  -1.706250  -1.162500
 -4.800000  -4.568750  -4.275000  -3.918750  -3.500000  -3.018750  -2.475000  -1.868750  -1.200000

Метод решения: седловая точка
x = 0.500000 = 1/2
y = 0.625000 = 5/8
N = -2.118750 = -339/160
Нижняя цена игры max(min) = -2.118750
Верхняя цена игры min(max) = -2.118750
Δ с предыдущим приближением: 0.000800
Седловые точки: (5, 6)
-----

```

Рисунок 7 – итерация численного решения №7

```

-----
Итерация 8
Разбиение: N = 9, матрица 10x10
Матрица игры:
  0.000000 -0.686420 -1.323457 -1.911111 -2.449383 -2.938272 -3.377778 -3.767901 -4.108642 -4.400000
 -0.138272 -0.725926 -1.264198 -1.753086 -2.192593 -2.582716 -2.923457 -3.214815 -3.456790 -3.649383
 -0.375309 -0.864198 -1.303704 -1.693827 -2.034568 -2.325926 -2.567901 -2.760494 -2.903704 -2.997531
 -0.711111 -1.101235 -1.441975 -1.733333 -1.975309 -2.167901 -2.311111 -2.404938 -2.449383 -2.444444
 -1.145679 -1.437037 -1.679012 -1.871605 -2.014815 -2.108642 -2.153086 -2.148148 -2.093827 -1.990123
 -1.679012 -1.871605 -2.014815 -2.108642 -2.153086 -2.148148 -2.093827 -1.990123 -1.837037 -1.634568
 -2.311111 -2.404938 -2.449383 -2.444444 -2.390123 -2.286420 -2.133333 -1.930864 -1.679012 -1.377778
 -3.041975 -3.037037 -2.982716 -2.879012 -2.725926 -2.523457 -2.271605 -1.970370 -1.619753 -1.219753
 -3.871605 -3.767901 -3.614815 -3.412346 -3.160494 -2.859259 -2.508642 -2.108642 -1.659259 -1.160494
 -4.800000 -4.597531 -4.345679 -4.044444 -3.693827 -3.293827 -2.844444 -2.345679 -1.797531 -1.200000

Метод решения: Браун-Робинсон
x = 0.487179 = 19/39
y = 0.584615 = 38/65
H = -2.118606 = -16112/7605
Нижняя цена игры max(min) = -2.153086
Верхняя цена игры min(max) = -2.108642
Δ с предыдущим приближением: 0.000144
Итераций Брауна-Робинсона: 65
Верхняя оценка цены игры: -2.077721
Нижняя оценка цены игры: -2.177398
Погрешность Брауна-Робинсона: 0.099677
-----

```

Рисунок 8 – итерация численного решения №8

```

-----
Итерация 9
Разбиение: N = 10, матрица 11x11
Матрица игры:
  0.000000 -0.620000 -1.200000 -1.740000 -2.240000 -2.700000 -3.120000 -3.500000 -3.840000 -4.140000 -4.400000
 -0.120000 -0.660000 -1.160000 -1.620000 -2.040000 -2.420000 -2.760000 -3.060000 -3.320000 -3.540000 -3.720000
 -0.320000 -0.780000 -1.200000 -1.580000 -1.920000 -2.220000 -2.480000 -2.700000 -2.880000 -3.020000 -3.120000
 -0.600000 -0.800000 -1.320000 -1.620000 -1.880000 -2.100000 -2.280000 -2.420000 -2.520000 -2.580000 -2.600000
 -0.960000 -1.260000 -1.520000 -1.740000 -1.920000 -2.060000 -2.160000 -2.220000 -2.240000 -2.220000 -2.160000
 -1.400000 -1.620000 -1.800000 -1.940000 -2.040000 -2.100000 -2.120000 -2.100000 -2.040000 -1.940000 -1.800000
 -1.920000 -2.060000 -2.160000 -2.220000 -2.240000 -2.220000 -2.160000 -2.060000 -1.920000 -1.740000 -1.520000
 -2.520000 -2.580000 -2.600000 -2.580000 -2.520000 -2.420000 -2.280000 -2.100000 -1.880000 -1.620000 -1.320000
 -3.200000 -3.180000 -3.120000 -3.020000 -2.880000 -2.700000 -2.480000 -2.220000 -1.920000 -1.580000 -1.200000
 -3.960000 -3.860000 -3.720000 -3.540000 -3.320000 -3.060000 -2.760000 -2.420000 -2.040000 -1.620000 -1.160000
 -4.800000 -4.620000 -4.400000 -4.140000 -3.840000 -3.500000 -3.120000 -2.700000 -2.240000 -1.740000 -1.200000

Метод решения: седловая точка
x = 0.500000 = 1/2
y = 0.600000 = 3/5
H = -2.120000 = -53/25
Нижняя цена игры max(min) = -2.120000
Верхняя цена игры min(max) = -2.120000
Δ с предыдущим приближением: 0.001394
Седловые точки: (6, 7)
-----

```

Рисунок 9 – итерация численного решения №9

```

-----
Итерация 10
Разбиение: N = 11, матрица 12x12
Матрица игры:
  0.000000 -0.565289 -1.097521 -1.596694 -2.062810 -2.495868 -2.895868 -3.262810 -3.596694 -3.897521 -4.165289 -4.400000
 -0.105785 -0.604959 -1.071074 -1.504132 -1.904132 -2.271074 -2.604959 -2.905785 -3.173554 -3.408264 -3.609917 -3.778512
 -0.277686 -0.710744 -1.110744 -1.477686 -1.811570 -2.112397 -2.380165 -2.614876 -2.816529 -2.985124 -3.120661 -3.223140
 -0.515702 -0.882645 -1.216529 -1.517355 -1.785124 -2.019835 -2.221488 -2.390083 -2.525620 -2.628099 -2.697521 -2.733884
 -0.819835 -1.120661 -1.388430 -1.623140 -1.824793 -1.993388 -2.128926 -2.231405 -2.300826 -2.337190 -2.340496 -2.310744
 -1.190083 -1.424793 -1.626446 -1.795041 -1.930579 -2.033058 -2.102479 -2.138843 -2.142149 -2.112397 -2.049587 -1.953719
 -1.626446 -1.795041 -1.930579 -2.033058 -2.102479 -2.138843 -2.142149 -2.112397 -2.049587 -1.953719 -1.824793 -1.662810
 -2.128926 -2.231405 -2.300826 -2.337190 -2.340496 -2.310744 -2.247934 -2.152066 -2.023140 -1.861157 -1.666116 -1.438017
 -2.697521 -2.733884 -2.737190 -2.707438 -2.644628 -2.548760 -2.419835 -2.257851 -2.062810 -1.834711 -1.573554 -1.279339
 -3.332231 -3.302479 -3.239669 -3.143802 -3.014876 -2.852893 -2.657851 -2.429752 -2.168595 -1.874380 -1.547107 -1.186777
 -4.033058 -3.937190 -3.808264 -3.646281 -3.451240 -3.223140 -2.961983 -2.667769 -2.340496 -1.980165 -1.586777 -1.160331
 -4.800000 -4.638017 -4.442975 -4.214876 -3.953719 -3.659504 -3.332231 -2.971901 -2.578512 -2.152066 -1.692562 -1.200000

Метод решения: Браун-Робинсон
x = 0.490397 = 383/781
y = 0.587708 = 459/781
H = -2.119122 = -676/319
Нижняя цена игры max(min) = -2.142149
Верхняя цена игры min(max) = -2.112397
Δ с предыдущим приближением: 0.000878
Итераций Брауна-Робинсона: 71
Верхняя оценка цены игры: -2.078408
Нижняя оценка цены игры: -2.177767
Погрешность Брауна-Робинсона: 0.099360
-----

```

Рисунок 10 – итерация численного решения №10

Окончательный результат получен на 364-й итерации решения, что соответствует $N=365$, то есть матрице размера 366×366 .

На последнем шаге:

- критерий останова выполнен;
- метод решения: седловая точка;
- последнее изменение:

$$|\Delta H| = 0.000010$$

Итоговое численное решение:

$$x \approx 0.498630, \quad y = 0.600000, \quad H \approx -2.120008$$

Сравним аналитическое и численное решения

Аналитически:

$$x^* = 0.500000, y^* = 0.600000, H^* = -2.120000$$

Численно:

$$x = 0.498630, y = 0.600000, H = -2.120008$$

Абсолютные погрешности:

$$|x - x^*| = 0.001370$$

$$|y - y^*| = 0.000000$$

$$|H - H^*| = 0.000008$$

Погрешность по цене игры очень мала, следовательно, численный метод дал результат, практически совпадающий с аналитическим.

Вывод:

В ходе лабораторной работы была исследована непрерывная выпукло-вогнутая антагонистическая игра на единичном квадрате.

Аналитически найдены следующие оптимальные стратегии и цена игры:

$$x^* = 0.5, \quad y^* = 0.6, \quad h = -2.12$$

Численное решение получалось методом аппроксимации функции выигрыша на сетке. Для каждой матрицы сначала проверялось наличие седловой точки, а при её отсутствии использовался метод Брауна-Робинсон. На 364-й итерации был выполнен критерий останова.

Сравнение результатов показало, что численный метод хорошо согласуется с аналитическим решением:

$$|H - H^*| = 0.000008$$

Следовательно, поставленная задача была решена корректно, а численный метод обеспечивает высокую точность приближения к точному решению.

Приложение (исходный код)

```
from __future__ import annotations
from dataclasses import dataclass
from fractions import Fraction
import numpy as np
from lab_1 import fmt, fr, pure_strategy_analysis, brown_robinson

OUTER_EPSILON = 1e-5
BR_EPSILON = 1e-1
MAX_N = 2000
BR_MAX_ITER = 10000
PRINT_FIRST_ITERATIONS = 10
PRINT_DETAILS_ITER = 10

def q(value: str) -> float:
    return float(Fraction(value))

@dataclass
class ContinuousSolution:
    x: float
    y: float
    h: float
    regime: str
    hx: float
    hy: float

@dataclass
class IterationResult:
    iteration_no: int
    N: int
    matrix: np.ndarray
    method: str
    x: float
    y: float
    h: float
    delta_prev: float | None
    v_lower: float
    v_upper: float
    saddle_points: list[tuple[int, int]]
    br_iterations: int | None = None
    br_gap: float | None = None
    br_upper: float | None = None
    br_lower: float | None = None

def kernel(x: float, y: float, a: float, b: float, c: float, d: float, e: float) -> float:
    return float(a * x * x + b * y * y + c * x * y + d * x + e * y)

def hx(x: float, y: float, a: float, c: float, d: float) -> float:
    return float(2 * a * x + c * y + d)

def hy(x: float, y: float, b: float, c: float, e: float) -> float:
    return float(c * x + 2 * b * y + e)

def build_matrix(a: float, b: float, c: float, d: float, e: float, N: int):
    grid = np.linspace(0.0, 1.0, N + 1)
    X, Y = np.meshgrid(grid, grid, indexing="ij")
    C = a * X * X + b * Y * Y + c * X * Y + d * X + e * Y
    return grid, C.astype(float)

def solve_continuous_game_exact(a: float, b: float, c: float, d: float, e: float,
                                tol: float = 1e-10) -> ContinuousSolution:
    candidates: list[ContinuousSolution] = []
    for x_case in ("0", "1", "i"):
        for y_case in ("0", "1", "i"):
            A = []
            rhs = []

            if x_case == "0":
                A.append([1.0, 0.0])
                rhs.append(0.0)
            elif x_case == "1":
                A.append([1.0, 0.0])
                rhs.append(1.0)
            else:
                pass
```

```

        A.append([2 * a, c])
        rhs.append(-d)

    if y_case == "0":
        A.append([0.0, 1.0])
        rhs.append(0.0)
    elif y_case == "1":
        A.append([0.0, 1.0])
        rhs.append(1.0)
    else:
        A.append([c, 2 * b])
        rhs.append(-e)

    A = np.array(A, dtype=float)
    rhs = np.array(rhs, dtype=float)

    try:
        x, y = np.linalg.solve(A, rhs)
    except np.linalg.LinAlgError:
        continue

    if not (-tol <= x <= 1 + tol and -tol <= y <= 1 + tol):
        continue

    x = min(max(float(x), 0.0), 1.0)
    y = min(max(float(y), 0.0), 1.0)

    dx = hx(x, y, a, c, d)
    dy = hy(x, y, b, c, e)

    ok = True

    if x_case == "0":
        ok = ok and dx <= tol
    elif x_case == "1":
        ok = ok and dx >= -tol
    else:
        ok = ok and (tol < x < 1 - tol) and abs(dx) <= 1e-7

    if y_case == "0":
        ok = ok and dy >= -tol
    elif y_case == "1":
        ok = ok and dy <= tol
    else:
        ok = ok and (tol < y < 1 - tol) and abs(dy) <= 1e-7

    if ok:
        candidates.append(
            ContinuousSolution(
                x=x,
                y=y,
                h=kernel(x, y, a, b, c, d, e),
                regime=f"x={x_case}, y={y_case}",
                hx=dx,
                hy=dy,
            )
        )

    if not candidates:
        raise RuntimeError("Не удалось найти аналитическое решение непрерывной игры.")

    candidates.sort(key=lambda s: (round(s.h, 12), s.x, s.y), reverse=True)
    return candidates[0]

def solve_matrix_game(
    C: np.ndarray,
    grid: np.ndarray,
    a: float,
    b: float,
    c: float,
    d: float,
    e: float,
    br_epsilon: float,
    br_max_iter: int,
) -> IterationResult:
    v_lower, v_upper, saddle_points = pure_strategy_analysis(C)

```

```

    if saddle_points:
        i, j = saddle_points[0]
        return IterationResult(
            iteration_no=0,
            N=len(grid) - 1,
            matrix=C.copy(),
            method="sedlo",
            x=float(grid[i]),
            y=float(grid[j]),
            h=float(C[i, j]),
            delta_prev=None,
            v_lower=v_lower,
            v_upper=v_upper,
            saddle_points=saddle_points,
        )

    history = brown_robinson(C, epsilon=br_epsilon, max_iter=br_max_iter, start_row=0,
start_col=0)
    last = history[-1]

    x_mix = np.asarray(last["x_est"], dtype=float)
    y_mix = np.asarray(last["y_est"], dtype=float)

    x_value = float(np.dot(x_mix, grid))
    y_value = float(np.dot(y_mix, grid))
    h_value = kernel(x_value, y_value, a, b, c, d, e)

    return IterationResult(
        iteration_no=0,
        N=len(grid) - 1,
        matrix=C.copy(),
        method="brown_robinson",
        x=x_value,
        y=y_value,
        h=h_value,
        delta_prev=None,
        v_lower=v_lower,
        v_upper=v_upper,
        saddle_points=[],
        br_iterations=int(last["k"]),
        br_gap=float(last["gap"]),
        br_upper=float(last["upper_best"]),
        br_lower=float(last["lower_best"]),
    )

def solve_iteratively(
    a: float,
    b: float,
    c: float,
    d: float,
    e: float,
    outer_epsilon: float,
    br_epsilon: float,
    max_N: int,
    br_max_iter: int,
):
    results: list[IterationResult] = []
    prev_h: float | None = None

    for N in range(2, max_N + 1):
        grid, C = build_matrix(a, b, c, d, e, N)
        res = solve_matrix_game(C, grid, a, b, c, d, e, br_epsilon, br_max_iter)
        res.iteration_no = N - 1
        res.delta_prev = None if prev_h is None else abs(res.h - prev_h)
        results.append(res)

        if prev_h is not None and res.delta_prev <= outer_epsilon:
            return results, True

        prev_h = res.h

    return results, False

def matrix_to_string(C: np.ndarray, digits: int = 6) -> str:
    rows = []

```

```

for row in C:
    rows.append(" ".join(f"{float(value):>{digits + 6}.{digits}f}" for value in row))
return "\n".join(rows)

def print_configuration(coeffs: dict[str, float]):
    print(f"\nВариант: 13")
    print("-" * 80)
    print("Рассматривается непрерывная антагонистическая игра на единичном квадрате")
    print("0 ≤ x ≤ 1, 0 ≤ y ≤ 1 с функцией выигрыша:")
    print(f"H(x, y) = a*x^2 + b*y^2 + c*x*y + d*x + e*y")
    print(
        f"a = {fr(coeffs['a'])}, b = {fr(coeffs['b'])}, c = {fr(coeffs['c'])}, d = {fr(coeffs['d'])}, e = {fr(coeffs['e'])}"
    )
    print(f"\nКритерий останова по внешней итерации: {OUTER_EPSILON}")
    print(f"Точность Брауна-Робинсон: {BR_EPSILON}")
    print(f"Максимальное N: {MAX_N}")
    print(f"Максимум итераций Брауна-Робинсон: {BR_MAX_ITER}")
    print("-" * 80)
    print()

def print_analytic_solution(sol: ContinuousSolution, a: float, b: float, c: float, d: float, e: float):
    print("Аналитическое решение непрерывной игры")
    print("-" * 80)
    print(f"H(x, y) = {fr(a)}*x^2 + {fr(b)}*y^2 + {fr(c)}*x*y + {fr(d)}*x + {fr(e)}*y")
    print(f"x* = {fmt(sol.x)} = {fr(sol.x)}")
    print(f"y* = {fmt(sol.y)} = {fr(sol.y)}")
    print(f"H(x*, y*) = {fmt(sol.h)} = {fr(sol.h)}")
    print(f"Активный режим: {sol.regime}")
    print(f"Hx(x*, y*) = {fmt(sol.hx)}")
    print(f"Hy(x*, y*) = {fmt(sol.hy)}")
    print("-" * 80)
    print()

def print_first_iterations_summary(results: list[IterationResult], limit: int = PRINT_FIRST_ITERATIONS):
    if PRINT_FIRST_ITERATIONS == 0:
        return 0

    print(f"Первые {min(limit, len(results))} итераций внешнего уточнения")
    print("-" * 120)
    print(f"{'Ит.':>4} | {'N':>4} | {'Размер':>8} | {'Метод':>16} | {'x':>10} | {'y':>10} | {'H':>12} | {'Δ_prev':>12}")
    print("-" * 120)

    for res in results[:limit]:
        size = f"{res.N + 1}x{res.N + 1}"
        method = "седло" if res.method == "sedlo" else "Брауна-Робинсон"
        delta_str = "---" if res.delta_prev is None else fmt(res.delta_prev)
        print(
            f"{res.iteration_no:>4} | {res.N:>4} | {size:>8} | {method:>16} | "
            f"{fmt(res.x):>10} | {fmt(res.y):>10} | {fmt(res.h):>12} | {delta_str:>12}"
        )

    print("-" * 120)
    print()

def print_iteration_details(results: list[IterationResult], limit: int = PRINT_DETAILS_ITER):
    if PRINT_DETAILS_ITER == 0:
        return 0

    print(f"Подробности по первым {min(limit, len(results))} итерациям")
    print("-" * 80)

    for res in results[:limit]:
        print(f"Итерация {res.iteration_no}")
        print(f"Разбиение: N = {res.N}, матрица {res.N + 1}x{res.N + 1}")
        print("Матрица игры:")
        print(matrix_to_string(res.matrix))
        print(f"Метод решения: {'седловая точка' if res.method == 'sedlo' else 'Браун-Робинсон'}")
        print(f"x = {fmt(res.x)} = {fr(res.x)}")
        print(f"y = {fmt(res.y)} = {fr(res.y)}")
        print(f"H = {fmt(res.h)} = {fr(res.h)}")
        print(f"Нижняя цена игры max(min) = {fmt(res.v_lower)}")
        print(f"Верхняя цена игры min(max) = {fmt(res.v_upper)}")

```

```

    if res.delta_prev is None:
        print("Δ с предыдущим приближением: ---")
    else:
        print(f"Δ с предыдущим приближением: {fmt(res.delta_prev)}")

    if res.method == "sedlo":
        saddles_str = ", ".join(f"({i + 1}, {j + 1})" for i, j in res.saddle_points)
        print(f"Седловые точки: {saddles_str}")
    else:
        print(f"Итераций Брауна-Робинсон: {res.br_iterations}")
        print(f"Верхняя оценка цены игры: {fmt(res.br_upper)}")
        print(f"Нижняя оценка цены игры: {fmt(res.br_lower)}")
        print(f"Погрешность Брауна-Робинсон: {fmt(res.br_gap)}")

    print("-" * 80)

print()

def print_final_summary(results: list[IterationResult], analytic: ContinuousSolution,
    outer_epsilon: float,
    converged: bool):
    last = results[-1]

    print("Итоговый результат")
    print("-" * 80)
    print(f"Критерий останова:  $|H_k - H_{(k-1)}| \leq \{outer\_epsilon\}$ ")
    print(f"Номер итерации, на которой получен итог: {last.iteration_no}")
    print(f"Разбиение:  $N = \{last.N\}$ , матрица  $\{last.N + 1\} \times \{last.N + 1\}$ ")
    print(f"Критерий останова {'выполнен' if converged else 'не выполнен, достигнут лимит по N'}")
    print(f"Метод на последнем шаге: {'седловая точка' if last.method == 'sedlo' else 'Браун-Робинсон'}")

    if last.delta_prev is not None:
        print(f"Последнее изменение  $|\Delta H| = \{fmt(last.delta\_prev)\}$ ")

    if last.method == "brown_robinson":
        print(f"Итераций Брауна-Робинсон на последнем шаге: {last.br_iterations}")
        print(f"Верхняя оценка цены игры: {fmt(last.br_upper)}")
        print(f"Нижняя оценка цены игры: {fmt(last.br_lower)}")
        print(f"Погрешность Брауна-Робинсон: {fmt(last.br_gap)}")

    print()
    print(f"Аналитическое решение:  $x^* = \{fmt(analytic.x)\}$ ,  $y^* = \{fmt(analytic.y)\}$ ,  $H^* = \{fmt(analytic.h)\}$ ")
    print(f"Итерационное решение:  $x = \{fmt(last.x)\}$ ,  $y = \{fmt(last.y)\}$ ,  $H = \{fmt(last.h)\}$ ")
    print(f" $|x - x^*| = \{fmt(abs(last.x - analytic.x))\}$ ")
    print(f" $|y - y^*| = \{fmt(abs(last.y - analytic.y))\}$ ")
    print(f" $|H - H^*| = \{fmt(abs(last.h - analytic.h))\}$ ")
    print("-" * 80)

def main():
    coeffs = {"a": q("-4"), "b": q("2"), "c": q("8"), "d": q("-4/5"), "e": q("-32/5")}
    a, b, c, d, e = coeffs["a"], coeffs["b"], coeffs["c"], coeffs["d"], coeffs["e"]
    print_configuration(coeffs)
    analytic = solve_continuous_game_exact(a, b, c, d, e)
    print_analytic_solution(analytic, a, b, c, d, e)
    results, converged = solve_iteratively(
        a=a,
        b=b,
        c=c,
        d=d,
        e=e,
        outer_epsilon=OUTER_EPSILON,
        br_epsilon=BR_EPSILON,
        max_N=MAX_N,
        br_max_iter=BR_MAX_ITER,
    )
    print_first_iterations_summary(results)
    print_iteration_details(results)
    print_final_summary(results, analytic, OUTER_EPSILON, converged)

if __name__ == "__main__":
    main()

```